

ConnectionsBot

Max Allworth-Miles, Owen Lutz, Gauransh Thapar

Connections

Create four groups of four!

ASTERISK	AXE	ANGEL	SURPRISE
SECRET	TRADEMARK	BONE	DOVE
DEGREE	PASSWORD	LAMB	KEYS
BABE	SKINS	EXPONENT	SPOILER

Mistakes Remaining: ● ● ● ●

[Shuffle](#) [Deselect All](#) [Submit](#)



Create four groups of four!

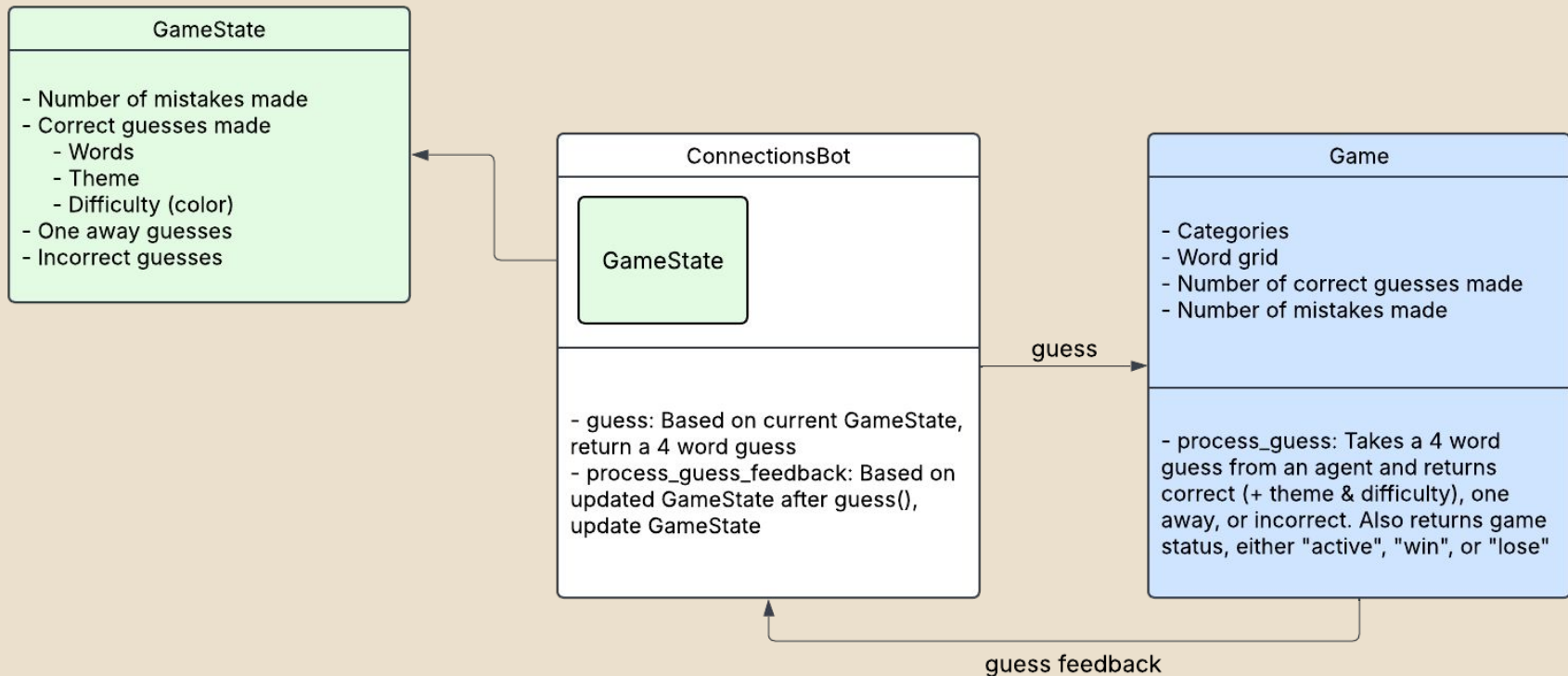
THINGS REPRESENTED IN SUPERSCRIPT ASTERISK, DEGREE, EXPONENT, TRADEMARK
SYMBOLS OF INNOCENCE ANGEL, BABE, DOVE, LAMB
THINGS YOU'RE NOT SUPPOSED TO REVEAL PASSWORD, SECRET, SPOILER, SURPRISE
SLANG FOR MUSICAL INSTRUMENTS AXE, BONE, KEYS, SKINS

[View Results](#)

Goal

Build an **intelligent system**
capable of solving Connections

System Design



Guessing Heuristics

Embedding Similarity

- Word embeddings: high dimensional vectors representative of a words meaning
- Method: Take the average cosine similarity between every pair of word vectors in every 4 word combinations, return the “most similar” (lowest mean cosine similarity) 4 words.
- Works well for groups of synonyms and groups of related words with a single dominant sense.
- Overall the best general use heuristic due to performance and efficiency.

WordNet

- Database of English words grouped by relationship (mostly interested in synonyms), with multiple distinct senses for words.
- Should perform better on groups that are synonyms of uncommon senses of words (ex: “bank” as in a river bank, not a financial institution).
- Method: For each word, find synonyms for each of its senses. If 3 synonyms are available words in the game, group into a guess.
- Didn't make any correct guesses using this method, removed from final program to save time

Character Insertion

- Connections categories often involve wordplay, this approach aims to solve a specific type
- Game Ex: “FOUNDER”, “SALON”, “SURGEON”, “TROT” → “FLOUNDER”, “SALMON”, “STURGEON”, “TROUT” (Fish minus a letter)
- Method: Insert characters into various positions in words. If resultant strings are valid english words, store them and take the average cosine similarity (borrowed from embedding similarity method) to find the 4 most similar modified words.

Homophone Similarity

- Homophone: A word that is pronounced the same as another word but differs in spelling
- Method: Find homophones of available words. Take the average cosine similarity between homophones (borrowed from embedding similarity method) to find the 4 most similar homophones, and return the original words.
- Game Ex: “AWL”, “NUN”, “PHEW”, “SUM” → “ALL”, “NONE”, “FEW”, “SOME” (Homophones of non-numeric amounts)

Subgroup Synonyms

- Detect hidden word relationships by finding meaningful subwords inside each word, such as prefixes, suffixes, or other substrings. Ex: Clap, Cutter have “cut” and “lop”
- Method: for each word, generate candidate subwords and score groups of four based on shared synonyms
- Either gets it right, or is useless.
- Computationally expensive

Low Hanging Fruit

- If there's only four left, pick them
- One-away, if we guessed a group which only had one wrong, focus on getting this group. In the aggregator, increase the bonus for groups with three of these words.

Ideas we tried!

We needed to combine the outputs of the heuristics in a useable way, so we tried a bunch of methods to solve the game problem.

- A Simple single guess method where we pick the best embedding match from among all the possible 4 words groups. We only update weights for other heuristics as we progress through solving easier levels.
 - Weighting heuristics for each guess (starting heavy on embedding) and returning the highest weight guess.
 - Rotating between the strategies based on events, starting with embedding.
- We got the idea of Event-Action model after trying this method

The first version of the bot was a strong baseline: it evaluated one guess at a time and chose the best locally scored option.

Ideas we tried!

- Then we moved to trying more complex ideas of combining guesses to make a set of 4 groups, and we assign each set a TOTAL score.
- Aggregate Method: Each heuristic returns a set of candidate groups, and the aggregator combines scores for identical guesses, adds bonuses for overlapping guesses, and chooses the group with the strongest total support.
- Weighted Heuristics per Group: score each candidate group with different heuristic mix formula depending on its position in the remaining solution path, then choose the full set with the strongest overall weighted score.

Weighted Heuristics per Group: We finally moved into this direction and came quite far (The bot could be improved a lot)

After making a lot of changes for performance/efficiency

- The Final bot does not just pick the best single guess.
- It generates many valid 4-word candidates for each group slot scored on the group's weightage formula.
- Then it creates a strongest shortlist for each group.
- It picks up the best from each group to form a valid set of 4 groups.
- Finally, it keeps going until it combines non-overlapping words/groups into full solution sets.
- It assigns a TOTAL score to each set, which is the sum of the scores of each of its groups combined. Builds a small ranked set list.
- The bot submits the next group from a set with highest TOTAL score.

Event - Action Model

How Feedback Changes the Strategy:

Correct guess:-

Keep the current Set, and remove the solved group. Return next guess.

One-away:-

Look for another set lower in the ranked hierarchy whose next submitted group differs from the one-away guess by exactly one word. Discard all other.

Incorrect:-

Discard the exact failed group. Look for other sets.

Strictly reject groups that share 3 words with a fully incorrect guess.

This prevents the bot from repeating a dead core.

Findings from an older version

1. Standalone first-guess quality by heuristic

Heuristic	Exact	One-away	Wrong
embedding	15 (75.0%)	3 (15.0%)	2 (10.0%)
phrase	3 (15.0%)	0 (0.0%)	0 (0.0%)
insertion	4 (20.0%)	3 (15.0%)	13 (65.0%)
homophone	0 (0.0%)	0 (0.0%)	0 (0.0%)

2. Exact standalone hits by color

Heuristic	YELLOW	GREEN	BLUE	PURPLE
embedding	9	2	2	2
phrase	0	0	0	3
insertion	1	1	0	2
homophone	0	0	0	0

3. One-away standalone hits by nearest color

Heuristic	YELLOW	GREEN	BLUE	PURPLE
embedding	1	1	1	0
phrase	0	0	0	0
insertion	1	0	0	2
homophone	0	0	0	0

Evidence:

```
embedding: raw=0.280, adjusted=0.280
partition: raw=0.000, adjusted=0.000
last_four: raw=999.000, adjusted=999000.000
one_away: raw=0.000, adjusted=0.000
insertion: raw=0.000, adjusted=0.000
homophone: raw=0.000, adjusted=0.000
hypernym: raw=0.000, adjusted=0.000
```

Chosen guess: ['mom', 'kayak', 'level', 'racecar']

```
Got correct group!: <Guesses.Guess object at 0x75d7d14f1820>, palindromes
Game 1 won! (18.541163444519043s)
snow  hail   rain   sleet
jazz  bucks  nets   heat
shift tab    return option
mom   kayak  level  racecar
```


Performance Tests

(these are old now, due to changes after presentation)

Win rate: 0.06

Games played: 250

Games won: 15

```
for, GROUP_PROFILE_WEIGHTS = {  
    "Group 1": {"embedding": 1.5, "phrase": 1.0, "insertion": 0.0, "homophone": 0.0},  
    "Group 2": {"embedding": 1.5, "phrase": 1.0, "insertion": 0.0, "homophone": 0.0},  
    "Group 3": {"embedding": 1.0, "phrase": 0.85, "insertion": 0.40, "homophone": 0.25},  
    "Group 4": {"embedding": 1.0, "phrase": 0.7, "insertion": 0.4, "homophone": 0.4},  
}
```

Win rate: 0.088

Games played: 250

Games won: 22

```
for, GROUP_PROFILE_WEIGHTS = {  
    "Group 1": {"embedding": 1.5, "phrase": 0.0, "insertion": 0.0, "homophone": 0.0},  
    "Group 2": {"embedding": 1.5, "phrase": 0.0, "insertion": 0.0, "homophone": 0.0},  
    "Group 3": {"embedding": 1.0, "phrase": 0.0, "insertion": 0.0, "homophone": 0.0},  
    "Group 4": {"embedding": 1.0, "phrase": 0.0, "insertion": 0.0, "homophone": 0.0},  
}
```

]

Design decisions for efficiency

Why Efficiency Matters?

- When we moved towards the sets ideas, a strong solver cannot afford to run every expensive heuristic on every candidate in a reasonable time.
- We found that embedding heuristic is very much cheaper than other heuristics, and the phrase heuristic gets a lot more complex as we increase the word checks.
- So we may have to add some global controls to tweak a balance between the Bot's performance in the game and the reasonable computational effort it makes for each game.

Design decisions for efficiency

The bot uses a filter for phrase heuristic:

- Cheap scoring first, using a `GLOBAL_CANDIDATE_PREFILTER`
- The embedding heuristic is used to make a shortlist of valid candidates
- Expensive heuristics like phrase only used on shortlisted candidates

We place a limit on the words that Phrase heuristic can match with, considering that it is the most expensive heuristic. (We tried, but discarded this)

- We placed 250 word limit on Phrase heuristic

Heuristic Gating by Weight: (This became very useful for pruning, so it's kept)

- A weight of 0 means the heuristic is skipped entirely for that group.

Future Work | *Now has been implemented after presentation*

- More guessing strategies! Connections is a very complex game, and covering more answer types could help performance
- We will tweak all the weights and other variables to find a sweet spot between the heuristics for each group.
 - Right now the Bot easily solves 1 or 2 groups, but struggles to move ahead.
- We will use an eligibility check for Homophone and Insertion heuristic and use the weights as a switch to admit them only if they make any sense.

Please go through the README!
It contains a lot more details about
the latest project's problem solving
aspect. *This is added after the presentation*

Thank You!